The Zen Model Family:

A Unified Architecture for Multimodal AI

Hanzo AI Research[1] **Zoo Labs Foundation**[2]

[1]Hanzo AI Inc. (Techstars '17) [2]Zoo Labs Foundation (501(c)(3))

research@hanzo.ai foundation@zoo.ngo

https://hanzo.ai/research/zen

February 2026

Abstract

We present the **Zen model family**, a comprehensive suite of foundation models spanning 600M to 480B parameters, built on the **Mixture of Distilled Experts (MoDE)** architecture. Zen models support text, vision, audio, code, and structured data within a unified transformer framework through modality-specific adapters that project heterogeneous inputs into a shared representation space. This report provides a complete technical account of the Zen architecture, training methodology, alignment procedures, and inference optimizations. We detail the MoDE architecture—a sparse mixture-of-experts design where expert networks are initialized via progressive distillation from dense teacher models, achieving 2.4× the effective capacity of equivalently-sized dense models while activating only 30–40% of parameters per token. We describe the four-stage training pipeline: (1) large-scale pre-training on 14.8T tokens across five modalities, (2) supervised fine-tuning on 8.2M curated instruction-response pairs, (3) RLHF and DPO alignment using 1.2M human preference comparisons, and (4) domain-specific adaptation for downstream applications. Across 42 benchmarks spanning language understanding (MMLU: 89.2%), mathematical reasoning (MATH: 78.6%), code generation (HumanEval: 92.1%), vision-language tasks (MMBench: 84.7%), and audio understanding (AudioBench: 81.3%), the Zen-72B model establishes new state-of-the-art results while requiring 2.1× less compute than comparable dense models. We further present inference optimizations—including speculative decoding, continuous batching, and quantization-aware training—that reduce serving costs by 3.8× without measurable quality degradation. All models from 600M to 72B are released under Apache 2.0.

Keywords: Foundation Models, Mixture of Experts, Multimodal AI, Language Models, Alignment, Inference Optimization

1 Introduction

The development of large language models (LLMs) has progressed rapidly from single-modality text generation to multimodal systems capable of understanding and producing text, images, audio, code, and structured data. This evolution reflects a fundamental insight: intelligence is not modality-specific but rather emerges from the ability to reason across diverse information types within a unified representational framework.

The Zen model family embodies this principle through a unified architecture that processes all modalities through a shared transformer backbone augmented with modality-specific adapters. Rather than training separate models for each modality and combining them post hoc, Zen learns cross-modal representations from the ground up, enabling emergent capabilities such as code generation from visual diagrams, audio description of images, and structured data extraction from natural language.

Central to the Zen architecture is the **Mixture of Distilled Experts (MoDE)** paradigm. Standard dense transformers activate all parameters for every input token, leading to computational costs that scale linearly with model size. Mixture-of-Experts (MoE) architectures (Shazeer et al., 2017; Fedus et al., 2022) address this by routing tokens to a subset of expert networks, achieving sublinear compute scaling. However, MoE models suffer from training instability, load imbalance, and expert collapse. MoDE resolves these issues through *progressive distillation*: expert networks are initialized from segments of a pre-trained dense model, inheriting stable representations that are then specialized through continued training with routing. This yields models that are both more parameter-efficient and more stable than conventional MoE architectures.

The Zen family spans six model sizes: 600M, 1.5B, 7B, 32B, 72B, and 480B parameters (with 200M, 500M, 2.3B, 10.6B, 24B, and 160B active parameters per token, respectively). This range enables deployment across the full spectrum of compute environments, from edge devices and mobile phones to multi-GPU server clusters.

This technical report makes the following contributions:

1. A complete description of the MoDE architecture, including the progressive distillation initialization procedure, routing mechanisms, and expert specialization dynamics.

2. Detailed documentation of the four-stage training pipeline, covering data curation, training infrastructure, and hyperparameter configurations across all model sizes.

3. Comprehensive evaluation across 42 benchmarks spanning five modalities, establishing state-of-the-art results on 31 benchmarks.

4. Novel inference optimization techniques that reduce serving costs by 3.8× while maintaining quality, enabling cost-effective deployment at scale.

5. Open release of all models from 600M to 72B under Apache 2.0, along with training code, evaluation harnesses, and serving infrastructure.

2 Background and Related Work

2.1 Large Language Models

The transformer architecture (Vaswani et al., 2017) has become the dominant paradigm for language modeling. Scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) established that model performance improves predictably with increases in model size, data volume, and compute. GPT-3 (Brown et al., 2020) demonstrated that sufficiently large language models exhibit in-context learning, and subsequent models have extended these capabilities to instruction following (Ouyang et al., 2022), code generation (Chen et al., 2021), and mathematical reasoning (Lewkowycz et al., 2022).

2.2 Mixture-of-Experts Models

Mixture-of-Experts (MoE) architectures replace dense feed-forward layers with collections of expert networks and a routing function that selects a subset of experts for each token (Shazeer et al., 2017). Switch Transformer (Fedus et al., 2022) simplified routing to single-expert selection, while GShard (Lepikhin et al., 2021) and ST-MoE (Zoph et al., 2022) explored scaling MoE to trillion-parameter models. More recently, Mixtral (Jiang et al., 2024) demonstrated that sparse MoE models can match dense model performance at significantly lower inference cost.

A persistent challenge in MoE training is expert collapse, where the routing function assigns most tokens to a small subset of experts. Auxiliary load-balancing losses (Fedus et al., 2022), expert choice routing (Zhou et al., 2022), and dropout regularization (Chen et al., 2023) have been proposed as mitigations, with varying effectiveness.

2.3 Multimodal Foundation Models

Multimodal models integrate information from multiple modalities—typically vision and language—into a unified system. Early approaches like CLIP (Radford et al., 2021) and ALIGN (Jia et al., 2021) learned joint vision-language representations through contrastive pre-training. Flamingo (Alayrac et al., 2022) and PaLM-E (Driess et al., 2023) demonstrated few-shot multimodal reasoning by conditioning language models on visual inputs. LLaVA (Liu et al., 2024) and similar architectures project visual features into the language model's embedding space through learned adapters.

Audio-language models (Gong et al., 2023; Tang et al., 2024) have followed a similar trajectory, adapting speech and audio encoders to interface with LLM backbones.

2.4 Alignment

Aligning language models with human intent and values is critical for safe and useful deployment. Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022; Bai et al., 2022a) trains a reward model on human preferences and optimizes the language model policy via PPO. Direct Preference Optimization (DPO) (Rafailov et al., 2024) simplifies this by directly optimizing the policy on preference data without a

separate reward model. Constitutional AI (Bai et al., 2022b) uses AI-generated critiques for self-improvement.

3 Architecture

3.1 Overview

The Zen architecture follows an encoder-free, decoder-only transformer design with modality adapters that project inputs from different modalities into a shared token space. The core model is a MoDE transformer that processes all modality tokens through the same self-attention and expert layers.

3.2 Mixture of Distilled Experts (MoDE)

Formulation. Each MoDE layer replaces the standard feed-forward network (FFN) with a collection of N expert networks and a gating function. Given an input token representation $\boldsymbol{x} \in \mathbb{R}^d$, the MoDE layer computes:

$$\text{MoDE}(\boldsymbol{x}) = \sum_{i=1}^{N} g_i(\boldsymbol{x}) \cdot E_i(\boldsymbol{x}) \tag{1}$$

where $E_i : \mathbb{R}^d \to \mathbb{R}^d$ is the i-th expert network (a two-layer FFN with SwiGLU activation) and $g_i(\boldsymbol{x})$ is the gating weight assigned to expert i for token $\boldsymbol{x}$.

Top-k Routing. The gating function selects the top-k experts with highest affinity for each token:

$$g_i(\boldsymbol{x}) = \begin{cases} \frac{\exp(\boldsymbol{w}_i^\top \boldsymbol{x})}{\sum_{j \in \text{Top-}k} \exp(\boldsymbol{w}_j^\top \boldsymbol{x})} & \text{if } i \in \text{Top-}k(\boldsymbol{W}\boldsymbol{x}) \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

where $\boldsymbol{W} = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_N] \in \mathbb{R}^{d \times N}$ is the router weight matrix and Top-k selects the indices of the k largest values. We use $k = 2$ for all model sizes, with N varying from 8 (600M) to 128 (480B).

Progressive Distillation Initialization. The key innovation of MoDE is the initialization procedure for expert networks. Rather than random initialization (which leads to training instability) or identical initialization (which leads to expert collapse), we initialize experts through *progressive distillation* from a pre-trained dense model:

1. **Dense Teacher Training:** Train a dense transformer with the same hidden dimension d on the full pre-training corpus.

2. **FFN Segmentation:** For each FFN layer, partition the intermediate dimension d_{ff} into N equal segments, where each segment forms the initial weights of one expert.

3. **Distillation Fine-tuning:** Train the MoDE model with the router on a subset of the pre-training data (10%), using a distillation loss that encourages the MoDE output to match the dense teacher:

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \mathcal{L}_{\text{LM}} + (1 - \alpha) \cdot \text{KL}\left(p_{\text{teacher}} \| p_{\text{MoDE}}\right) \tag{3}$$

   with $\alpha = 0.7$ decaying to 1.0 over 5000 steps.

4. **Expert Specialization:** Continue training on the full dataset, allowing experts to specialize while maintaining diversity through auxiliary losses.

Load Balancing. We use a combination of auxiliary losses to prevent expert collapse:

$$\mathcal{L}_{\text{balance}} = \beta_1 \cdot \mathcal{L}_{\text{importance}} + \beta_2 \cdot \mathcal{L}_{\text{load}} + \beta_3 \cdot \mathcal{L}_{\text{diversity}} \tag{4}$$

where $\mathcal{L}_{\text{importance}}$ penalizes variance in expert utilization across the batch, $\mathcal{L}_{\text{load}}$ penalizes deviation from uniform load distribution, and $\mathcal{L}_{\text{diversity}}$ encourages experts to develop distinct specializations by penalizing high cosine similarity between expert weight matrices. We set $\beta_1 = 0.01$, $\beta_2 = 0.01$, $\beta_3 = 0.001$.

3.3 Attention Mechanism

All Zen models use Grouped Query Attention (GQA) (Ainslie et al., 2023) with Rotary Position Embeddings (RoPE) (Su et al., 2024) extended to support long contexts via YaRN (Peng et al., 2024).

GQA Configuration. We use a head ratio of 8:1 (query heads to key-value heads), reducing the KV cache memory by 8× compared to standard multi-head attention while preserving quality:

$$\text{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V} \tag{5}$$

where $\boldsymbol{Q} \in \mathbb{R}^{n_q \times d_k}$, $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n_{kv} \times d_k}$, $n_q = 8 \cdot n_{kv}$, and $\boldsymbol{M}$ is the causal attention mask.

Extended Context. Base RoPE supports 8,192 tokens. We extend to 128K tokens using YaRN with a scale factor of 16 and NTK-aware interpolation:

$$\theta_i' = \theta_i \cdot \left(\frac{s \cdot L_{\text{new}}}{L_{\text{base}}}\right)^{-2i/d} \tag{6}$$

where s is the NTK scale, $L_{\text{new}} = 131072$, and $L_{\text{base}} = 8192$. Effective context utilization is validated by achieving >95% accuracy on the Needle-in-a-Haystack test at 128K context length.

3.4 Modality Adapters

Each input modality is processed by a dedicated adapter that converts raw inputs into token sequences compatible with the transformer backbone.

Vision Adapter. The vision adapter uses a SigLIP (Zhai et al., 2023) encoder (ViT-SO400M/14) to extract patch features from images at multiple resolutions. Features are projected through a two-layer MLP with GELU activation into the transformer's embedding space:

$$\boldsymbol{v}_i = \text{MLP}_{\text{vision}}(\text{SigLIP}(I, r_j)) \in \mathbb{R}^d \tag{7}$$

where $r_j \in \{224, 448, 896\}$ are the resolution levels. Dynamic resolution selection uses the image's native resolution, processing at the closest supported level. A special `<image>` token triggers the adapter, and visual tokens are interleaved with text tokens in the input sequence.

Audio Adapter. The audio adapter processes waveforms through a Whisper-Large (Radford et al., 2023) encoder, extracting 80-dimensional log-Mel spectrogram features at 16kHz. The encoder output is downsampled by a factor of 4 through a strided convolutional layer and projected into the embedding space:

$$\boldsymbol{a}_i = \text{MLP}_{\text{audio}}\left(\text{Conv1D}_{\downarrow 4}(\text{Whisper}(\boldsymbol{w}))\right) \in \mathbb{R}^d \tag{8}$$

This yields approximately 25 tokens per second of audio, enabling processing of up to 85 minutes of audio within the 128K context window.

Code Adapter. Rather than a separate encoder, the code adapter applies structural tokenization that preserves syntactic information. Source code is parsed into an AST using tree-sitter, and special tokens mark structural boundaries (function definitions, control flow, scope changes). This structural tokenization improves code understanding by 4.2% on HumanEval compared to plain text tokenization.

Structured Data Adapter. Tables and structured data are serialized using a linearization scheme that preserves row-column relationships through positional tokens:

$$\texttt{<row:}i\texttt{><col:}j\texttt{>}\ v_{ij}\ \texttt{<sep>} \tag{9}$$

This representation enables the model to perform table QA, data transformation, and schema reasoning without specialized table encoders.

3.5 Model Configurations

Table 1 summarizes the architecture configurations across all Zen model sizes.

Table 1: Zen model family configurations. "Active" denotes parameters activated per token.

Model	Total	Active	Layers	d	Experts	Context
Zen-600M	600M	200M	24	1024	8	32K
Zen-1.5B	1.5B	500M	28	1536	8	32K
Zen-7B	7B	2.3B	32	3584	16	128K
Zen-32B	32B	10.6B	48	5120	32	128K
Zen-72B	72B	24B	64	6144	64	128K
Zen-480B	480B	160B	96	8192	128	128K

Table 2: Pre-training data composition by modality and source.

Modality	Source	Tokens	Weight
Text	Web (filtered CommonCrawl)	8.2T	40%
	Books & academic papers	1.5T	12%
	Wikipedia & encyclopedias	0.3T	5%
	Conversational data	0.4T	3%
Code	GitHub (permissive licenses)	2.1T	18%
	Documentation & READMEs	0.3T	2%
Vision	Image-text pairs (LAION, CC)	1.2T	10%
	Video-text pairs	0.2T	3%
Audio	Speech & music transcripts	0.4T	4%
Structured	Tables, JSON, SQL	0.2T	3%
Total		**14.8T**	100%

4 Training

The Zen training pipeline consists of four stages: pre-training, supervised fine-tuning (SFT), alignment, and domain adaptation. We describe each stage in detail.

4.1 Stage 1: Pre-training

Data Composition. Pre-training uses 14.8 trillion tokens across five modalities:

Data Quality Pipeline. Raw data undergoes a multi-stage quality pipeline:

1. **Deduplication:** MinHash-based near-duplicate detection with Jaccard threshold 0.8, removing 34% of web data.
2. **Quality Scoring:** A classifier trained on 50K human-rated examples assigns quality scores. Documents below the 30th percentile are discarded.
3. **Safety Filtering:** Removal of personally identifiable information (PII), toxic content, and CSAM using a combination of regex patterns, classifier-based detection, and hash-matching against known harmful content databases.

4. **Domain Balancing:** Upsampling of underrepresented high-quality domains (math, science, legal) and downsampling of overrepresented low-quality domains (SEO content, auto-generated text).

Tokenizer. We use a byte-pair encoding (BPE) tokenizer with a vocabulary of 152,064 tokens, including 1,024 special tokens for modality markers, structural annotations, and reserved future use. The tokenizer is trained on a representative subset of the pre-training data with language-aware sampling to ensure adequate coverage of CJK characters, mathematical notation, and code syntax.

Training Objective. Pre-training uses the standard causal language modeling objective:

$$\mathcal{L}_{\text{LM}} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t|x_{<t}) \tag{10}$$

augmented with the MoDE load balancing loss (Section 3.2):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{LM}} + \mathcal{L}_{\text{balance}} \tag{11}$$

Optimization. We use AdamW (Loshchilov and Hutter, 2019) with the following hyperparameters:

- Learning rate: 3×10^{-4} (Zen-600M) to 1.5×10^{-4} (Zen-480B), with cosine decay to 10% of peak
- Warm-up: 2000 steps (linear)
- Batch size: 4M tokens (Zen-600M) to 32M tokens (Zen-480B)
- Weight decay: 0.1
- Gradient clipping: 1.0
- $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$

Training Schedule. Pre-training follows a multi-phase schedule:

1. **Phase 1 (0–60% tokens):** Standard pre-training on the full data mix.
2. **Phase 2 (60–85% tokens):** Increased weight on high-quality data (books, papers, curated web) and code.
3. **Phase 3 (85–100% tokens):** Long-context training with sequences up to 128K tokens, gradually increasing context length from 8K to 128K.

4.2 Stage 2: Supervised Fine-Tuning

After pre-training, models undergo supervised fine-tuning (SFT) on 8.2 million curated instruction-response pairs.

Data Sources.

- **Human-written instructions (2.1M):** Collected from professional annotators across 12 task categories (QA, summarization, creative writing, analysis, coding, math, etc.).
- **Multi-turn conversations (1.8M):** Naturally occurring conversations with 3–15 turns, covering information seeking, task execution, and collaborative problem-solving.
- **Code instructions (1.5M):** Programming tasks spanning 38 languages, including debugging, refactoring, code review, and generation.
- **Multimodal instructions (1.3M):** Vision-language (800K), audio-language (300K), and structured data (200K) instruction-response pairs.
- **Safety-focused data (0.8M):** Examples demonstrating appropriate refusals, nuanced handling of sensitive topics, and calibrated uncertainty expression.
- **Synthetic augmentation (0.7M):** High-quality synthetic data generated by the Zen-480B model and validated by human reviewers.

Training. SFT uses the same causal LM objective but masks the loss on instruction tokens, computing loss only on response tokens:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{R}|} \sum_{t \in \mathcal{R}} \log p_\theta(x_t | x_{<t}) \tag{12}$$

where $\mathcal{R}$ is the set of response token positions. We train for 3 epochs with learning rate 2×10^{-5} and batch size 512.

4.3 Stage 3: Alignment

Alignment combines RLHF and DPO to optimize for human preferences while maintaining broad capabilities.

Preference Data Collection. We collect 1.2 million pairwise preference comparisons from a team of 450 trained annotators. Each comparison presents two model responses to the same prompt, and annotators select the preferred response based on helpfulness, honesty, and harmlessness. Inter-annotator agreement (Fleiss' κ) averages 0.73 across all categories.

Reward Model. A reward model R_ψ is trained on the preference data using the Bradley-Terry model:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(R_\psi(x, y_w) - R_\psi(x, y_l) \right) \right] \tag{13}$$

where y_w and y_l are the preferred and dispreferred responses. The reward model is initialized from the SFT checkpoint and trained for 1 epoch with learning rate 1×10^{-5}.

PPO Training. We optimize the policy using PPO (Schulman et al., 2017) with the following objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] - \beta \cdot \text{KL}(\pi_\theta \| \pi_{\text{ref}}) \tag{14}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$ is the probability ratio, $\hat{A}_t$ is the advantage estimate, $\epsilon = 0.2$ is the clip range, and $\beta = 0.02$ controls the KL penalty against the reference policy.

DPO Refinement. After PPO, we apply DPO (Rafailov et al., 2024) as a refinement step using a fresh set of 200K preference pairs:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \tag{15}$$

with $\beta = 0.1$. DPO provides 1.2% additional improvement on reward model scores beyond PPO alone.

4.4 Stage 4: Domain Adaptation

Specialized variants (Zen-Coder, Zen-Medical, Zen-Reasoning, etc.) undergo additional domain-specific training using curated data and task-specific objectives. Each variant begins from the aligned Zen checkpoint and applies LoRA (Hu et al., 2022) adapters ($r = 64$, $\alpha = 128$) to minimize catastrophic forgetting while enabling rapid specialization.

4.5 Training Infrastructure

Hardware. Training is conducted on clusters of NVIDIA H100 GPUs:

- Zen-600M to Zen-7B: 64 H100 GPUs (8 nodes $\times$ 8 GPUs)
- Zen-32B: 256 H100 GPUs (32 nodes $\times$ 8 GPUs)
- Zen-72B: 512 H100 GPUs (64 nodes $\times$ 8 GPUs)
- Zen-480B: 2048 H100 GPUs (256 nodes $\times$ 8 GPUs)

Parallelism Strategy. We combine four parallelism dimensions:

- **Data Parallelism (DP):** ZeRO Stage 3 (Rajbhandari et al., 2020) across nodes
- **Tensor Parallelism (TP):** Megatron-style (Shoeybi et al., 2019) within each node (8-way)
- **Pipeline Parallelism (PP):** 4-way for Zen-72B, 8-way for Zen-480B
- **Expert Parallelism (EP):** MoDE experts distributed across TP ranks

Compute Budget. Total pre-training compute:

Table 3: Training compute budget by model size.

Model	GPUs	Days	GPU-hours	FLOPs
Zen-600M	64	3	4,608	1.2×10^{20}
Zen-1.5B	64	7	10,752	5.8×10^{20}
Zen-7B	64	21	32,256	4.1×10^{21}
Zen-32B	256	18	110,592	3.2×10^{22}
Zen-72B	512	28	344,064	1.4×10^{23}
Zen-480B	2048	42	2,064,384	2.1×10^{24}

Training Stability. We employ several techniques to ensure stable training:

- **Gradient accumulation with loss spike detection:** Gradients with norm $> 5\times$ the running average are discarded and the step is retried.
- **Checkpoint averaging:** Final models are exponential moving averages (EMA) of the last 1000 steps ($\gamma = 0.9999$).
- **Mixed precision:** BF16 for forward/backward passes, FP32 for optimizer states and gradient accumulation.
- **Automatic restart:** Training recovers from hardware failures by rolling back to the nearest checkpoint (saved every 100 steps).

5 Inference Optimization

Efficient inference is critical for cost-effective deployment. We describe three complementary optimization techniques.

5.1 Speculative Decoding

Speculative decoding (Leviathan et al., 2023) uses a smaller draft model to generate candidate tokens that are verified by the target model in parallel. We use the Zen-600M model as the drafter for Zen-72B, achieving an acceptance rate of 78% (average 3.1 tokens accepted per speculation step):

$$\text{Speedup} = \frac{k \cdot \gamma}{1 + (1 - \gamma) \cdot c} \tag{16}$$

where $k = 4$ is the speculation length, $\gamma = 0.78$ is the acceptance rate, and c is the relative cost of the draft model ($c = 0.03$ for Zen-600M vs Zen-72B). This yields a $2.4\times$ speedup for greedy decoding.

5.2 Quantization-Aware Training

We apply quantization-aware training (QAT) during the final 5% of pre-training, enabling INT4 weight quantization with minimal quality loss:

$$\hat{\boldsymbol{W}} = s \cdot \text{clip}\left(\left\lfloor \frac{\boldsymbol{W}}{s} \right\rceil, -8, 7\right), \quad s = \frac{\max(|\boldsymbol{W}|)}{7} \tag{17}$$

where $\lfloor \cdot \rceil$ denotes round-to-nearest. QAT preserves 99.4% of FP16 quality on MMLU while reducing model size by 4× and memory bandwidth requirements proportionally.

GPTQ and AWQ Compatibility. Models are also compatible with post-training quantization methods. We provide official GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) quantized checkpoints for all model sizes.

5.3 Continuous Batching and KV Cache Optimization

We implement continuous batching with PagedAttention (Kwon et al., 2023) for efficient serving:

- **Paged KV cache:** KV cache memory is managed in 4KB pages, eliminating fragmentation and enabling memory sharing across requests with common prefixes.
- **Prefix caching:** System prompts and repeated context are cached across requests, reducing first-token latency by 40–60% for applications with shared context.
- **Dynamic batch scheduling:** Requests are batched by sequence length and priority, maximizing GPU utilization while meeting latency SLAs.

Combined, these optimizations reduce per-token serving cost by 3.8× compared to naive FP16 inference, making Zen-72B economically viable for consumer-facing applications.

6 Evaluation

We evaluate the Zen model family across 42 benchmarks spanning language understanding, reasoning, code generation, multimodal understanding, and safety.

Table 4: Language understanding benchmarks. Zen-72B results are 5-shot unless noted.

Benchmark	Zen-7B	Zen-32B	Zen-72B	Prev. SOTA
MMLU	72.4	83.1	89.2	87.5
MMLU-Pro	58.3	71.2	78.4	75.1
ARC-Challenge	88.2	93.7	96.1	95.4
HellaSwag	82.1	88.4	91.3	90.1
WinoGrande	78.6	84.2	87.8	86.3
TruthfulQA	52.3	64.8	71.2	68.7
BBH	68.4	78.3	84.6	82.1

Table 5: Mathematical and scientific reasoning benchmarks.

Benchmark	Zen-7B	Zen-32B	Zen-72B	Prev. SOTA
GSM8K	82.1	90.4	95.2	93.8
MATH	51.3	68.2	78.6	74.2
GPQA	32.1	41.8	52.3	48.7
ARC-AGI (public)	18.4	29.6	38.2	34.1
SciQ	91.2	95.8	97.3	96.1
MathVista	48.2	62.1	71.8	67.4

6.1 Language Understanding

6.2 Mathematical and Scientific Reasoning

6.3 Code Generation

6.4 Multimodal Understanding

6.5 Safety and Alignment

6.6 Scaling Analysis

We observe consistent log-linear scaling of benchmark performance with model size across all evaluation categories. The MoDE architecture provides a favorable compute-performance trade-off: Zen-72B (24B active parameters) matches or exceeds the performance of dense 70B models while requiring 2.1× less inference compute.

7 Ablation Studies

7.1 MoDE vs. Standard MoE

We compare MoDE initialization against standard MoE training (random initialization, identical initialization) on the Zen-7B architecture:

MoDE initialization provides the best combination of quality, training stability (zero restarts required), and expert utilization (94% of experts receive meaningful traffic).

Table 6: Code generation benchmarks (pass@1 unless noted).

Benchmark	Zen-7B	Zen-32B	Zen-72B	Prev. SOTA
HumanEval	72.6	85.4	92.1	89.4
HumanEval+	65.8	79.2	87.3	84.1
MBPP	74.2	84.6	90.8	88.2
MBPP+	62.3	74.8	82.1	79.6
MultiPL-E (avg)	58.4	72.1	81.6	78.3
SWE-Bench Lite	21.3	34.2	42.8	38.6

Table 7: Vision-language and audio-language benchmarks.

Benchmark	Zen-7B	Zen-32B	Zen-72B	Prev. SOTA
MMBench	68.3	78.2	84.7	81.3
MMMU	42.1	54.3	62.8	58.1
MathVista	48.2	62.1	71.8	67.4
DocVQA	78.4	86.2	91.3	88.7
ChartQA	72.1	82.8	88.4	85.2
AudioBench	61.2	74.3	81.3	77.8
LibriSpeech (WER)	4.2	3.1	2.4	2.8

7.2 Number of Experts

The optimal expert count for 7B parameters is 16, balancing model quality and inference throughput.

7.3 Modality Adapter Design

The SigLIP + MLP adapter achieves the best quality-parameter trade-off, outperforming the more complex Q-Former while adding fewer parameters.

8 Discussion

8.1 MoDE Advantages

The progressive distillation initialization provides three key advantages over standard MoE:

1. **Training stability:** By inheriting representations from a converged dense model, MoDE avoids the early-training instability that frequently requires restarts in standard MoE training.

2. **Expert diversity:** Segmenting the dense FFN ensures experts begin with different specializations, avoiding the collapse pathology.

3. **Data efficiency:** MoDE reaches target quality with 20% fewer tokens than standard MoE initialization, likely because the distilled initialization provides a better starting point for specialization.

Table 8: Safety evaluation benchmarks.

Benchmark	Zen-7B	Zen-72B	Prev. SOTA
ToxiGen (lower is better)	8.2	3.1	4.8
BBQ Bias (accuracy)	78.4	88.2	85.1
BOLD (toxicity, lower)	5.1	2.3	3.2
HarmBench (ASR, lower)	12.3	4.7	6.8
MT-Bench	8.2	9.1	8.8
AlpacaEval 2.0 (LC win%)	32.1	51.8	48.2

Table 9: Compute efficiency comparison (MMLU accuracy vs. inference FLOPs per token).

Model	MMLU (%)	FLOPs/token	Efficiency
Dense-70B (baseline)	87.1	1.4×10^{11}	1.0×
Zen-72B (MoDE)	89.2	6.7×10^{10}	2.1×
Zen-32B (MoDE)	83.1	2.9×10^{10}	4.6×

8.2 Limitations

- **Expert memory overhead:** While active parameters are reduced, total parameters (and thus model storage) remain large. This limits on-device deployment to smaller variants.
- **Routing overhead:** The routing computation adds approximately 3% overhead per layer, which is amortized for large batch sizes but can be significant for latency-sensitive single-request inference.
- **Hallucination:** Despite alignment, Zen models occasionally generate factually incorrect statements, particularly for rare or recent knowledge.
- **Multimodal integration:** Audio understanding lags behind vision-language capabilities, reflecting the smaller proportion of audio data in pre-training.

8.3 Societal Impact

Large language models have both positive and negative societal implications. Positive applications include education, accessibility (translation, text-to-speech), scientific research, and software development assistance. Risks include misinformation generation, bias amplification, and job displacement. We mitigate these through safety training (Section 4.3), content filtering in our API, and usage policies that prohibit harmful applications. The open release of smaller model variants enables the research community to study and improve upon our safety measures.

9 Conclusion

We presented the Zen model family, a comprehensive suite of multimodal foundation models built on the Mixture of Distilled Experts (MoDE) architecture. MoDE addresses

Table 10: Ablation: MoDE initialization vs. alternatives (Zen-7B, trained on 2T tokens).

Init Method	MMLU	HumanEval	Stability	Expert Util.
Random init	66.8	64.2	2 restarts	71%
Identical init	68.1	66.4	0 restarts	42%
Hash routing	69.4	68.1	0 restarts	100%
MoDE (ours)	**72.4**	**72.6**	0 restarts	94%

Table 11: Effect of expert count at fixed 7B total parameters.

N (experts)	Active Params	MMLU	Throughput (tok/s)
4	3.5B	71.2	2,800
8	2.8B	71.8	2,450
16	2.3B	72.4	2,100
32	2.1B	72.1	1,750
64	2.0B	71.6	1,200

the core challenges of mixture-of-experts training—instability, expert collapse, and inefficient specialization—through progressive distillation from dense teacher models. The resulting models achieve state-of-the-art performance across 42 benchmarks while requiring significantly less inference compute than comparably-sized dense models.

The four-stage training pipeline (pre-training, SFT, alignment, domain adaptation) produces models that are not only capable but also safe and aligned with human values. Inference optimizations including speculative decoding, quantization-aware training, and continuous batching reduce serving costs by 3.8×, enabling practical deployment at scale.

By releasing models from 600M to 72B parameters under Apache 2.0, we aim to democratize access to state-of-the-art AI capabilities while fostering a vibrant ecosystem of research and applications. Models, code, and evaluation harnesses are available at https://github.com/hanzoai/zen.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023.

Alayrac, J.-B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., et al. Flamingo: A visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, 2022.

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie,

Table 12: Ablation of vision adapter design on MMBench (Zen-7B).

Adapter Design	MMBench	Params Added
Linear projection	62.1	2M
2-layer MLP	66.8	8M
Q-Former (6 layers)	67.2	120M
SigLIP + MLP (ours)	**68.3**	28M

A., Mirhoseini, A., McKinnon, C., et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pp. 1877–1901, 2020.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training and serving system of foundation models: A comprehensive survey. *arXiv preprint arXiv:2401.02643*, 2023.

Driess, D., Xia, F., Sajjadi, M. S., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., et al. PaLM-E: An embodied multimodal language model. In *International Conference on Machine Learning*, 2023.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

Gong, Y., Luo, H., Liu, A. H., Karlinsky, L., and Glass, J. Listen, think, and understand. *arXiv preprint arXiv:2305.10790*, 2023.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Jia, C., Yang, Y., Xia, Y., Chen, Y.-T., Parekh, Z., Pham, H., Le, Q., Sung, Y.-H., Li, Z., and Duerig, T. Scaling up visual and vision-language representation learning

with noisy text supervision. In *International Conference on Machine Learning*, pp. 4904–4916, 2021.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, pp. 611–626, 2023.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286, 2023.

Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., et al. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*, 2022.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, T., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *MLSys*, 2024.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2024.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, pp. 27730–27744, 2022.

Peng, B., Quesnelle, J., Fan, H., and Shippole, E. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pp. 8748–8763, 2021.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pp. 28492–28518, 2023.

Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2024.

Rajbhandari, S., Rasley, J., Rber, O., and He, Y. ZeRO: Memory optimizations toward training trillion parameter models. In *International Conference on High Performance Computing*, pp. 1–16, 2020.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Tang, C., Yu, W., Sun, G., Chen, X., Tan, T., Li, W., Lu, L., Ma, Z., and Zhang, C. SALMONN: Towards generic hearing abilities for large language models. In *International Conference on Learning Representations*, 2024.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.

Zhai, X., Mustafa, B., Kolesnikov, A., and Beyer, L. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 11975–11986, 2023.

Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A. M., Le, Q. V., Laudon, J., et al. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems*, 2022.

Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., and Fedus, W. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.

A Complete Benchmark Results

Table 13: Complete results across all 42 benchmarks for all model sizes.

Benchmark	600M	1.5B	7B	32B	72B	480B
MMLU	32.1	48.3	72.4	83.1	89.2	92.1
MMLU-Pro	21.4	34.2	58.3	71.2	78.4	82.3
ARC-C	48.2	62.4	88.2	93.7	96.1	97.4
HellaSwag	42.1	58.3	82.1	88.4	91.3	93.2
WinoGrande	52.3	62.1	78.6	84.2	87.8	89.4
TruthfulQA	38.2	42.1	52.3	64.8	71.2	74.3
BBH	28.4	41.2	68.4	78.3	84.6	88.1
GSM8K	12.3	32.4	82.1	90.4	95.2	97.1
MATH	4.2	14.8	51.3	68.2	78.6	84.2
HumanEval	18.3	38.4	72.6	85.4	92.1	95.3
MBPP	22.1	42.3	74.2	84.6	90.8	93.7
MMBench	–	28.4	68.3	78.2	84.7	88.2

B Training Loss Curves

All models exhibit smooth loss convergence with no significant instabilities, confirming the effectiveness of the MoDE initialization procedure. The Zen-72B model reaches a final training loss of 1.72 nats on the validation set after processing 14.8T tokens. Loss decreases monotonically with model size at fixed token count, consistent with established scaling laws.

C Tokenizer Details

The Zen tokenizer is a BPE tokenizer trained on a multilingual corpus with the following characteristics:

- Vocabulary size: 152,064 tokens
- Special tokens: 1,024 reserved

- Byte fallback: All byte values are in the vocabulary, ensuring lossless encoding of any UTF-8 text
- Compression ratio: 3.8 characters per token (English), 1.2 characters per token (Chinese), 2.1 characters per token (code)
- Training data: 100B characters sampled proportionally from the pre-training data mix